

SecurityRequirement

S2 - chiusura semantica del metamodel e corpus facial-access camera

Dal SpecializedRequirement astratto alla specializzazione di sicurezza concreta

Stato. Chiusura semantica S2 per il baseline corrente. I probe hanno chiuso il delta minimo di **SecurityRequirement**; restano fuori Finding, Base Analysis, STRIDE, risk, control e il meccanismo strutturale di provenance.

Baseline repository. nballestriero/documentation-driven-threat-analysis, commit b9bbe690ba9f73e38f75ff9e7d8d36a88a16f49a.

Obiettivo. Formalizzare il minimo delta semantico di SecurityRequirement rispetto a SpecializedRequirement e conservarne la closure evidence sul corpus telecamera già usato in S1.

Indice

1	Scope S2 e baseline ereditato	2
2	Due assi da non confondere: tipo e ownership	2
2.1	Gerarchia di tipo	2
2.2	Catena documentale concreta	2
3	Domanda fondamentale S2	2
4	Sanity check terminologico esterno	4
5	Definizione semantica chiusa	4
6	Forma minima: quattro alternative	4
7	SecurityProperty come delta strutturale	5
7.1	Perche' non basta inferire sempre la property dal testo	5
7.2	Perche' cardinalita' uno	5
8	Security failure mode: semantica obbligatoria, campo non ancora necessario	6
9	Cause neutrality	6
9.1	Failure mode non e' causa	6
10	Corpus facial-access camera: baseline funzionale	7
10.1	MacroRequirement	7
10.2	Decision di placement	7
10.3	FunctionalRequirement	7
11	SR-3.4-C come SecurityRequirement: Confidentiality	8
11.1	Regression checks	8
12	SR-3.4-I come SecurityRequirement: Integrity	8
13	SR-3.4-P come SecurityRequirement: Authorized Provenance	9
14	Split test: una property per Requirement	9
15	Cause mutation sullo stesso SecurityRequirement	10
16	Caso limite: perdita dati per guasto	10
16.1	Guasto con solo failure funzionale ordinario	10
16.2	Guasto che viola una property di sicurezza governata	11
17	Availability probe e confine con reliability	11

18 Negative controls S2	11
18.1 NC-SEC-01 - Functional correctness	11
18.2 NC-SEC-02 - Control/realization leakage	11
18.3 NC-SEC-03 - Attack description	12
18.4 NC-SEC-04 - Generic quality concern	12
19 Architecture mutation regression	12
20 Whole-chain validation	12
21 Cosa e' completato con la closure S2	13
22 Stato epistemico S2 dopo il corpus	13
23 Invarianti S2 chiusi	14
24 Reopen criteria e confine successivo	15
25 Riferimenti terminologici di controllo	15

1 Scope S2 e baseline ereditato

S2 non riapre i layer già chiusi. Assume il contratto documentale raggiunto fino a S1.5:

```
GovernedDocument
|
+-- MacroRequirement
+-- Decision
'-- Requirement [abstract]
    |
    +-- FunctionalRequirement
    '-- SpecializedRequirement [abstract]
```

con il contratto comune:

```
Requirement [abstract]
  extends GovernedDocument
  normativeClause : NormativeClause [1..*]
```

Ogni Requirement è esso stesso una obbligazione normativa governata; tutte le sue clausole devono costituire una sola unità normativa coerente. Una obbligazione indipendentemente evolvibile o assessabile deve essere modellata come Requirement separato.

Per SpecializedRequirement restano chiusi: parent FR singolo, normative strengthening, composizione congiuntiva, removal test, no duplicazione della correttezza funzionale, subordinate positive action, autonomia operativa come boundary verso FR, realization separation e parent dependence.

S2 boundary

S2 determina che cosa renda uno SpecializedRequirement specificamente un SecurityRequirement e quale sia il suo contratto minimo aggiuntivo. La struttura dell'analisi che lo motiva resta downstream e non deve essere incorporata nel requisito per convenienza.

2 Due assi da non confondere: tipo e ownership

La specializzazione di tipo e la catena di ownership sono due viste diverse.

2.1 Gerarchia di tipo

2.2 Catena documentale concreta

Non esiste quindi una istanza intermedia SpecializedRequirement tra FR e SecurityRequirement. Lo stesso oggetto concreto è contemporaneamente SecurityRequirement, SpecializedRequirement, Requirement e GovernedDocument.

3 Domanda fondamentale S2

Fundamental question

Dato uno SpecializedRequirement già valido rispetto a un singolo FunctionalRequirement, quale informazione e quale semantica aggiuntiva devono essere presenti affinché esso sia specificamente un SecurityRequirement, senza incorporare la causa analitica, l'attacco, il finding o il controllo che ne motivano o realizzano l'obbligazione?

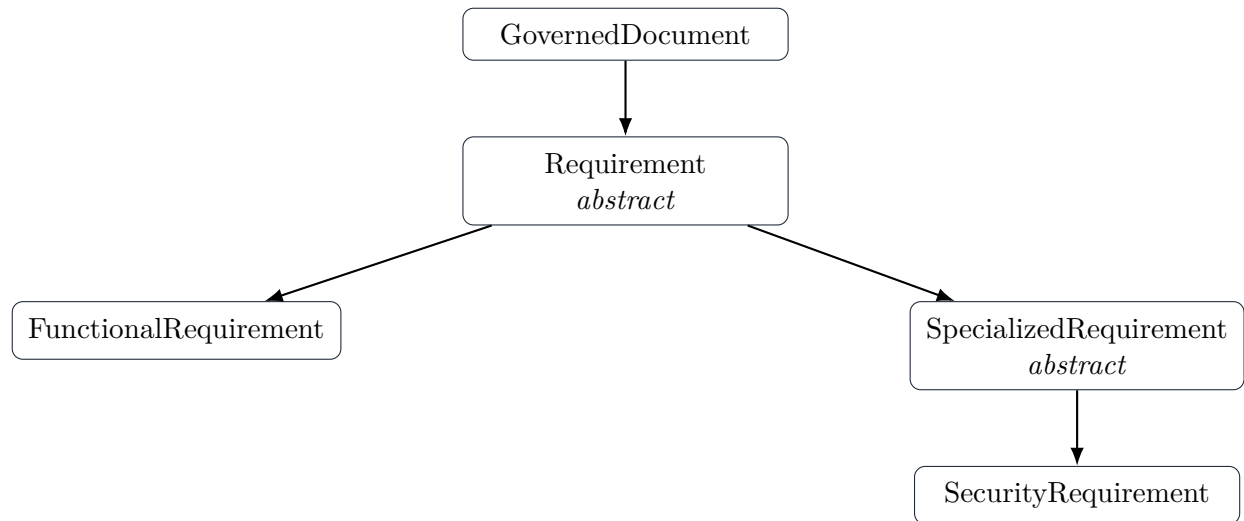


Figura 1: Gerarchia IS-A. SecurityRequirement e' un tipo concreto di SpecializedRequirement, non un documento figlio aggiuntivo.

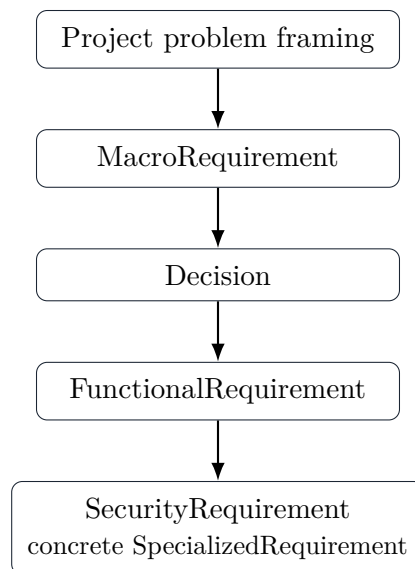


Figura 2: Ownership/decomposition dei documenti concreti nel caso security.

La domanda separa quattro concetti:

1. **obbligazione governata**: il SecurityRequirement stesso;
2. **security property**: la proprieta' di sicurezza che il requisito intende preservare;
3. **security failure mode**: lo stato/comportamento che renderebbe osservabile la perdita di quella proprieta' nel contesto del parent FR;
4. **cause**: attacco, errore, guasto o altro evento capace di condurre al failure mode.

S2 verifica quali di questi elementi appartengano alla struttura intrinseca del documento e quali debbano restare nella semantica normativa o nell'analysis model.

4 Sanity check terminologico esterno

La definizione DDTA non viene derivata da una tassonomia esterna, ma due osservazioni NIST sono utili come controllo di non-contraddizione:

- NIST descrive il rischio di sicurezza dei sistemi in termini di perdita di confidentiality, integrity o availability e riconosce che ulteriori proprietà come authenticity e accountability possono essere rilevanti;
- la metodologia di risk assessment NIST SP 800-30 contempla sorgenti/eventi adversarial e non-adversarial e dispone template distinti per entrambe le famiglie.

Queste osservazioni non impongono il nostro metamodel, ma supportano una scelta importante: non definire *security* come sinonimo di *attacco intenzionale*.

Scope discipline

La semantica SecurityRequirement può essere cause-neutral senza obbligare la tesi a offrire coverage esaustivo di fault, safety, reliability o resilience. Una tecnica di analisi specifica può deliberatamente coprire solo una parte delle cause; la limitazione di coverage appartiene all'analysis method, non alla definizione del requisito.

5 Definizione semantica chiusa

CLOSED - S2 definition

Un *SecurityRequirement* è uno *SpecializedRequirement* la cui obbligazione normativa preserva una singola proprietà di sicurezza governata rilevante per il parent *FunctionalRequirement* e rende identificabile, nelle proprie normative clause, il security failure mode che costituirebbe la perdita o violazione di tale proprietà nel contesto del parent FR.

La definizione mantiene quindi:

$$\text{SecR} \subseteq \text{SR}$$

ed eredita da S1:

$$\text{Effective}(F) = F \wedge S_1 \wedge \dots \wedge S_n.$$

Il fatto che S_i sia un SecurityRequirement non modifica la semantica congiuntiva: la specializzazione Security determina il concern e il contratto aggiuntivo, non una nuova regola di composizione.

6 Forma minima: quattro alternative

Sono state confrontate quattro forme candidate.

Forma	Struttura	Problema da testare
A	nessun nuovo campo	Property e failure mode devono essere ricostruiti interamente dalle clause.

Forma	Struttura	Problema da testare
B	<code>protectedSecurityProperty</code> [1]	La property diventa interrogabile; failure mode resta single-source nelle clause.
C	<code>securityFailureMode</code> [1]	Il failure mode e' strutturato, ma la classificazione/coverage per property resta implicita.
D	property + failure mode	Massima esplicitezza, ma possibile duplicazione tra failure mode strutturato e contenuto normativo.

I probe successivi eliminano le forme ridondanti o semanticamente insufficienti. La closure S2 seleziona la forma B: `protectedSecurityProperty` [1] e' strutturata, mentre il failure mode rimane obbligatoriamente esplicito nelle normative clause. Le forme A e C sono insufficienti per il contratto desiderato; la forma D rimane non necessaria finche' una identita' strutturale separata del failure mode non sia forzata da un controesempio.

7 SecurityProperty come delta strutturale

7.1 Perche' non basta inferire sempre la property dal testo

In un singolo documento una property come Confidentiality e' spesso ovvia leggendo la clause. DDTA deve pero' poter sostenere anche query governate quali:

```
show all SecurityRequirements protecting Confidentiality
which SecurityProperties are represented under FR-3.4?
which requirements independently protect Integrity?
```

Se la property non ha una rappresentazione strutturata, il tool dovrebbe reinterpretare continuamente il linguaggio naturale per rispondere. Questo indebolirebbe stable identity, single-source governed knowledge e tool non-authority.

CLOSED - structural delta

```
SecurityRequirement
  extends SpecializedRequirement

  protectedSecurityProperty : SecurityProperty [1]
```

`SecurityProperty` e' per ora una governed reference concettuale. S2 non chiude un enum universale ne' una tassonomia esaustiva.

7.2 Perche' cardinalita' uno

S1.5 ha chiuso il coherent-unit invariant. Se Confidentiality e Integrity possono essere introdotte, soddisfatte, revisionate o ritirate indipendentemente, non devono essere aggregate nello stesso Requirement soltanto perche' entrambe sono security-related.

```
SR-3.4-C : SecurityRequirement
  protectedSecurityProperty = Confidentiality
```

```
SR-3.4-I : SecurityRequirement
  protectedSecurityProperty = Integrity
```

La cardinalita' uno non implica che la futura tassonomia SecurityProperty debba essere piatta o mutuamente esclusiva. Una property puo' essere raffinata o specializzata; S2 chiude soltanto un riferimento governante singolo per ogni Requirement coerente.

8 Security failure mode: semantica obbligatoria, campo non ancora necessario

Un SecurityRequirement povero come:

```
Ensure confidentiality.
```

non e' sufficiente per DDTA. La property identifica il concern, ma non dice che cosa significhi concretamente perderlo nel comportamento del parent FR.

S2 richiede quindi failure-mode explicitness:

CLOSED - SEC-01 Failure-mode explicitness

Le normative clause di un SecurityRequirement MUST rendere identificabile lo stato, comportamento o risultato che costituirebbe la violazione della protectedSecurityProperty nel contesto del parent FunctionalRequirement.

Per ora non viene introdotto:

```
securityFailureMode : SecurityFailureMode [1]
```

come campo obbligatorio separato. Nei probe correnti il failure mode coincide con informazione normativa gia' necessaria alla clause; duplicarlo come secondo source of truth non aggiunge responsabilita' semantica indipendente.

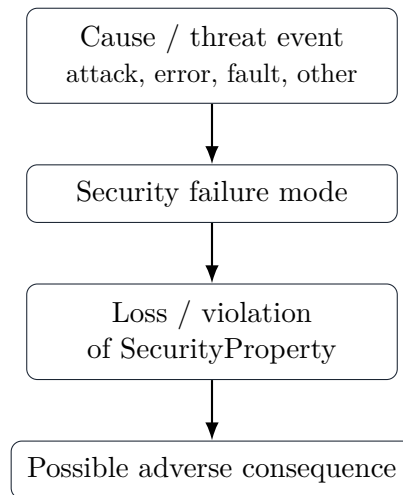
OPEN - structural extraction

Un futuro analysis/tool layer potra' richiedere una rappresentazione strutturata del failure mode. S2 la introdurra' solo se un controesempio dimostra che la sola explicitness nelle clause non consente governance, analisi o tracciabilita' senza perdita.

9 Cause neutrality

9.1 Failure mode non e' causa

La catena concettuale e':



Il SecurityRequirement governa la proprietà e il failure mode; la causa spiega come si possa arrivare a quel failure mode e appartiene al futuro analysis model.

CLOSED - SEC-02 Cause neutrality

L'identità e la validità normativa di un SecurityRequirement MUST NOT dipendere da una singola causa adversarial o non-adversarial. Attacco, errore umano, misconfiguration, software fault o hardware fault possono motivare review dello stesso failure mode senza diventare parte necessaria dell'identità del requisito.

Cause neutrality non significa che tutte le cause siano equivalenti nell'analisi, nella likelihood o nei controlli. Significa soltanto che il requisito non viene ridefinito ogni volta che cambia la spiegazione causale della stessa perdita di proprietà.

10 Corpus facial-access camera: baseline funzionale

Il corpus riusa la decomposizione S1 senza introdurre una nuova architettura.

10.1 MacroRequirement

MR-0003 - Riconoscimento facciale per la verifica della persona. Il progetto possiede la responsabilità macro di produrre evidenza di riconoscimento facciale usabile nel processo di verifica della persona.

10.2 Decision di placement

D-3.3 - Recognition placement. CameraSubsystem acquisisce l'evidenza al punto di accesso; un RecognitionProcessor separato esegue localmente/remotamente rispetto alla camera il riconoscimento governato dal corpus.

Nel probe corrente la capture deve quindi essere consegnata dal CameraSubsystem al RecognitionProcessor.

10.3 FunctionalRequirement

FR-3.4 - Deliver RecognitionCapture. Per ogni RecognitionCapture destinata alla valutazione remota, il CameraSubsystem MUST consegnare la capture al RecognitionProcessor

associandola alla corretta **RecognitionRequest**; una consegna non completata **MUST NOT** essere rappresentata come completata con successo.

Questa e' correttezza funzionale ordinaria. Associare la capture alla richiesta sbagliata non viene spostato in SecurityRequirement soltanto perche' potrebbe essere sfruttato da un attaccante.

11 SR-3.4-C come SecurityRequirement: Confidentiality

```
id:
  SR-3.4-C

type:
  SecurityRequirement

parentFunctionalRequirement:
  FR-3.4

protectedSecurityProperty:
  Confidentiality

normativeClause:
  Durante la consegna governata da FR-3.4, il contenuto
  della RecognitionCapture MUST NOT diventare intelligibile
  a un soggetto non autorizzato ad accedere a quella capture.
```

Il failure mode e' identificabile senza un campo duplicato:

```
RecognitionCapture becomes intelligible
or disclosed to an unauthorized subject.
```

11.1 Regression checks

- **Parent ownership:** esattamente FR-3.4 - PASS.
- **Removal:** senza SR-3.4-C la delivery resta funzione riconoscibile, ma perde confidentiality - PASS.
- **Security specificity:** property esplicita e failure mode contestualizzato - PASS.
- **Realization separation:** TLS, payload encryption o altra strategia equivalente non cambiano il Requirement - PASS.

12 SR-3.4-I come SecurityRequirement: Integrity

```
id:
  SR-3.4-I

type:
  SecurityRequirement

parentFunctionalRequirement:
  FR-3.4

protectedSecurityProperty:
```

Integrity**normativeClause:**

Una RecognitionCapture alterata dopo l'acquisizione
MUST NOT essere accettata come la capture governata
associata alla relativa RecognitionRequest.

Il failure mode e':

altered RecognitionCapture is accepted
as the governed capture for the request.

Il requisito non prescrive hash, MAC, firma, protocollo o layer di protezione. Queste sono possibili realization choices downstream.

13 SR-3.4-P come SecurityRequirement: Authorized Provenance

```
id:
  SR-3.4-P

type:
  SecurityRequirement

parentFunctionalRequirement:
  FR-3.4

protectedSecurityProperty:
  Authenticity / Authorized Provenance    [candidate label]

normativeClause 1:
  Una RecognitionCapture MUST NOT essere accettata come
  evidenza valida se non e' stabilito che la sua origine
  e' autorizzata a fornire evidenza per la relativa request.

normativeClause 2:
  Il mancato accertamento di una origine autorizzata
  MUST NOT essere rappresentato come accettazione riuscita.
```

Le due clause restano un Requirement soltanto se acceptance condition e failure outcome sono parti inseparabili della stessa obbligazione di authorized provenance.

OPEN - SecurityProperty vocabulary

Il label **Authenticity / Authorized Provenance** e' deliberatamente candidato. S2 non decide ancora se Authorized Provenance sia una SecurityProperty autonoma, un refinement di Authenticity o una governed property definita dal progetto. La tassonomia non deve essere chiusa solo per accomodare il corpus.

14 Split test: una property per Requirement

Consideriamo la proposta errata:

```
SR-3.4-X
protectedSecurityProperty = {Confidentiality, Integrity}
clause 1 = capture MUST NOT be disclosed
```

clause 2 = capture MUST NOT be altered and accepted

Le due obbligazioni possono essere violate, implementate, assessate e revisionate indipendentemente. Il coherent-unit invariant S1.5 forza quindi lo split:

SR-3.4-C -> Confidentiality
SR-3.4-I -> Integrity

CLOSED - SEC-03 Single governing security property

Ogni SecurityRequirement possiede esattamente un riferimento governante a SecurityProperty. Se due property esprimono obbligazioni indipendentemente evolvibili/assessable, devono essere modellate come SecurityRequirement distinti.

15 Cause mutation sullo stesso SecurityRequirement

Prendiamo SR-3.4-C. Manteniamo invariati property, failure mode e normative clause, cambiando soltanto la causa ipotizzata.

Mutation	Cause	Esito sul requisito
A	malicious network observation/interception	SR-3.4-C invariato; la causa e' adversarial.
B	endpoint misconfiguration exposing content	SR-3.4-C invariato; la causa e' non-adversarial/accidentale.
C	software defect exposing capture	SR-3.4-C invariato; cambia l'origine causale, non il failure mode.
D	hardware replacement/disposal path exposes stored copy	SR-3.4-C puo' continuare a esprimere confidentiality se il trattamento ricade nel parent/applicability governato; eventuale cambio di parent richiede review, non auto-migrazione.

La mutazione supporta SEC-02: l'attacco non e' prerequisito ontologico del SecurityRequirement.

16 Caso limite: perdita dati per guasto

La frase "perdita dati dovuta a rottura" non basta da sola a classificare un Requirement come security.

16.1 Guasto con solo failure funzionale ordinario

Se un componente si rompe e FR-3.4 non completa la delivery, la clausola funzionale gia' richiede che la consegna fallita non venga rappresentata come successo. Questo rimane FR correctness, anche se il guasto e' grave.

16.2 Guasto che viola una property di sicurezza governata

Se invece il progetto governa una property di Availability/Integrity e il failure mode e', per esempio, perdita permanente di un record necessario oppure indisponibilita' non tollerabile di una capability autorizzata, un evento non-adversarial puo' motivare un SecurityRequirement relativo a quella property.

Il criterio non e':

```
Was there an attacker?
```

ma:

```
Is there a governed SecurityProperty whose loss is made
explicit by this Requirement and whose failure mode is
separate from ordinary functional correctness?
```

Lo stesso fenomeno puo' essere simultaneamente rilevante per reliability/resilience. S2 non pretende di rendere SecurityRequirement l'unica classificazione possibile di ogni guasto.

17 Availability probe e confine con reliability

Per stressare il modello senza estendere il corpus canonico, consideriamo un probe illustrativo sotto un FR che processa richieste valide:

```
protectedSecurityProperty:
  Availability

normativeClause:
  Repeated invalid submissions MUST NOT permanently starve
  governed valid RecognitionRequests from receiving the
  processing service owned by the parent FR.
```

Il failure mode e' starvation permanente delle richieste valide. Una causa potrebbe essere adversarial (denial-of-service intenzionale) oppure non-adversarial (client difettoso che genera richieste invalide). Il Requirement puo' restare semanticamente uguale.

Negative control: un generico "the service must never fail" non e' automaticamente SecurityRequirement; puo' essere reliability/performance e manca di una property/failure mode sufficientemente governati.

18 Negative controls S2

18.1 NC-SEC-01 - Functional correctness

```
RecognitionCapture MUST be associated with the correct
RecognitionRequest.
```

E' necessaria alla funzione di delivery stessa. Rimane in FR-3.4 anche se una associazione errata puo' produrre conseguenze di sicurezza.

18.2 NC-SEC-02 - Control/realization leakage

```
RecognitionCapture MUST be protected with TLS 1.3.
```

Nella forma corrente e' una scelta di realizzazione/Decision candidata, non la semantica minima del SecurityRequirement. Il requisito deve esprimere property e failure mode senza prescrivere incidentalmente il meccanismo.

18.3 NC-SEC-03 - Attack description

`An attacker MUST NOT perform a MITM attack.`

Non definisce in modo sufficiente quale property del parent FR debba essere preservata, quale comportamento costituisca failure e quale obbligo governato debba valere. E' una descrizione causale/threat-oriented, non un SecurityRequirement completo.

18.4 NC-SEC-04 - Generic quality concern

`Delivery should be reliable and fast.`

Non e' una obligation sufficientemente assessable e non identifica una SecurityProperty. Potrebbe generare futuri Performance/QualityRequirement, ma non e' SecurityRequirement per sola rilevanza operativa.

19 Architecture mutation regression

S2 deve conservare le mutation gia' superate da S1.

Mutation	FR/SecR	Interpretazione
Ethernet → Wi-Fi	FR-3.4 e SecR possono restare invariati dopo review	Cambia il mezzo, non necessariamente l'obbligo.
Transport external → project-owned	puo' emergere un nuovo FR di transport capability; SecR su FR-3.4 possono restare	Ownership della realizzazione non trasferisce ownership normativa.
Recognition remote → local to camera	FR-3.4 scompare; i SecR figli vengono riesaminati e possono diventare non-applicable/retired	Conferma parent dependence e no auto-migration.
Raw capture → opaque reference	review locale dei SecR interessati	Il modello richiede effective governed context, ma non introduce in S2 una generic affects relation.

20 Whole-chain validation

Il corpus consente ora di attraversare l'intero metamodel della documentazione governata fino alla sicurezza:

Il passaggio non richiede Threat, Attack, Finding, Control o AnalysisRecord dentro la semantica dei documenti. Questi concetti potranno spiegare coverage, motivazione, plausibilita', risk e realization downstream.

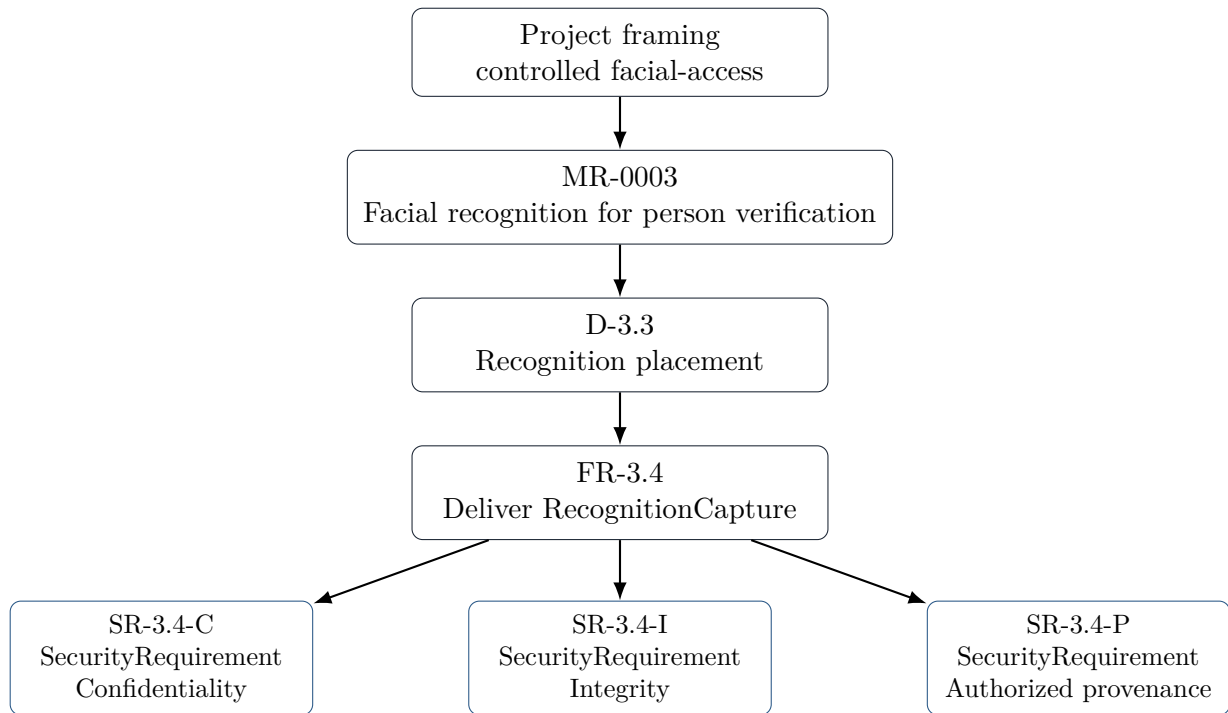


Figura 3: Catena documentale concreta. I tre SecurityRequirement sono SpecializedRequirement concreti indipendenti dello stesso FR.

21 Cosa e' completato con la closure S2

Con la closure degli invarianti S2, e' dichiarato completo per il baseline corrente il **governed documentation metamodel through SecurityRequirement**:

```

Project problem framing [method precondition]
  -> MacroRequirement
    -> Decision
      -> FunctionalRequirement
        -> SpecializedRequirement [abstract]
          -> SecurityRequirement
  
```

Questo non equivale ancora al full DDTA metamodel. Restano da formalizzare almeno il lato analitico e la sua integrazione:

```

Governed documentation
  -> Base Analysis / analyzable representation
  -> Analysis execution
  -> Finding / analytical result
  -> governed feedback / provenance
  -> requirement or other document change
  
```

La separazione e' intenzionale: S2 chiude la destinazione normativa senza anticipare la struttura che produrrà o motivara' i cambiamenti.

22 Stato epistemico S2 dopo il corpus

Stato	Elementi
CLOSED	Baseline MR/Decision/FR; Requirement abstraction S1.5; SpecializedRequirement S1; parent singolo; coherent-unit; provenance distinta dalla semantica.
CLOSED	SecurityRequirement IS-A SpecializedRequirement; definizione property-centered; protectedSecurityProperty : SecurityProperty [1]; failure-mode explicitness; cause neutrality; single governing security property.
CANDIDATE	Forma finale del label/property per Authorized Provenance; possibilità di usare SecurityProperty come governed reference estensibile; availability probe come generality evidence.
OPEN	Tassonomia/gerarchia di SecurityProperty; eventuale futura rappresentazione strutturata di SecurityFailureMode; exact L2 representation; effective governed context.
DEFERRED	Attack, ThreatActor, ThreatEvent, Incident, Vulnerability, Finding, Risk, Control, Base Analysis/BAE, STRIDE formalization, analysis coverage, provenance/change-event model, verification evidence.

23 Invarianti S2 chiusi

- S2-CLOSED-1. Subtype.** Ogni SecurityRequirement e' uno SpecializedRequirement e ne conserva tutti gli invarianti.
- S2-CLOSED-2. Security-property anchoring.** Ogni SecurityRequirement riferisce esattamente una SecurityProperty governante.
- S2-CLOSED-3. Failure-mode explicitness.** Le normative clause rendono identificabile che cosa costituisce perdita/violazione della property nel parent FR.
- S2-CLOSED-4. Cause neutrality.** L'identita' normativa non dipende da una specifica causa adversarial o non-adversarial.
- S2-CLOSED-5. Attack non-necessity.** La presenza di un attacco noto non e' prerequisito per la validita' del requisito.
- S2-CLOSED-6. Single coherent security obligation.** Property e clause devono formare una sola unita' normativa; property indipendenti richiedono Requirement distinti.
- S2-CLOSED-7. Functional preservation.** La rimozione del SecurityRequirement lascia riconoscibile il parent FR, pur perdendo la property di sicurezza.
- S2-CLOSED-8. No functional theft.** Una condizione di ordinaria correttezza funzionale non viene spostata in SecurityRequirement per sola security relevance.
- S2-CLOSED-9. No attack-as-requirement.** Descrivere attacker, technique o threat event non sostituisce property, failure mode e obligation.
- S2-CLOSED-10. Realization separation.** Control, protocollo, algoritmo, componente o layer non sono parte necessaria della semantica SecurityRequirement.
- S2-CLOSED-11. Method neutrality.** La semantica SecurityRequirement non richiede STRIDE o altra tecnica specifica.

S2-CLOSED-12. Provenance separation. La causa analitica che motiva il requisito non viene incorporata come singolo analysis id intrinseco.

24 Reopen criteria e confine successivo

S2 e' chiuso per il baseline corrente e deve essere riaperto se emerge almeno uno dei seguenti controesempi:

- un SecurityRequirement valido richiede necessariamente due SecurityProperty indipendenti nello stesso Requirement;
- un failure mode non puo' essere governato senza una identita' strutturale separata dalle normative clause;
- la causa adversarial/non-adversarial modifica inevitabilmente l'identita' normativa anche mantenendo invariati property e failure outcome;
- il modello non riesce a distinguere in modo stabile security da ordinary functional correctness o da quality/reliability;
- un caso security concreto forza Attack, Finding, Risk o Control dentro il core del requisito.

Closure boundary

S2 chiude il governed documentation metamodel through SecurityRequirement. I successivi microstep non devono riaprire questa semantica salvo controesempio concreto. La tassonomia SecurityProperty, la possibile identita' strutturale del failure mode e il lato analitico DDTA restano rispettivamente OPEN o DEFERRED secondo lo stato epistemico sopra.

25 Riferimenti terminologici di controllo

- NIST SP 800-30 Rev. 1, *Guide for Conducting Risk Assessments*: threat sources/events e template di assessment adversarial e non-adversarial.
- NIST CSRC Glossary, *Information System-Related Security Risk*: rischio derivante dalla perdita di confidentiality, integrity o availability.
- NIST CSRC Glossary, *Security*: confidentiality, integrity e availability come proprieta' centrali; ulteriori property possono essere rilevanti a seconda del contesto.

Questi riferimenti sono usati solo come sanity check terminologico. Le cardinalita', i boundary e gli invarianti DDTA del presente documento derivano dai probe del metamodel e non sono attribuiti a NIST.